

Pilhas em C: A Arquitetura LIFO

Do conceito físico à manipulação de memória em Vetores.

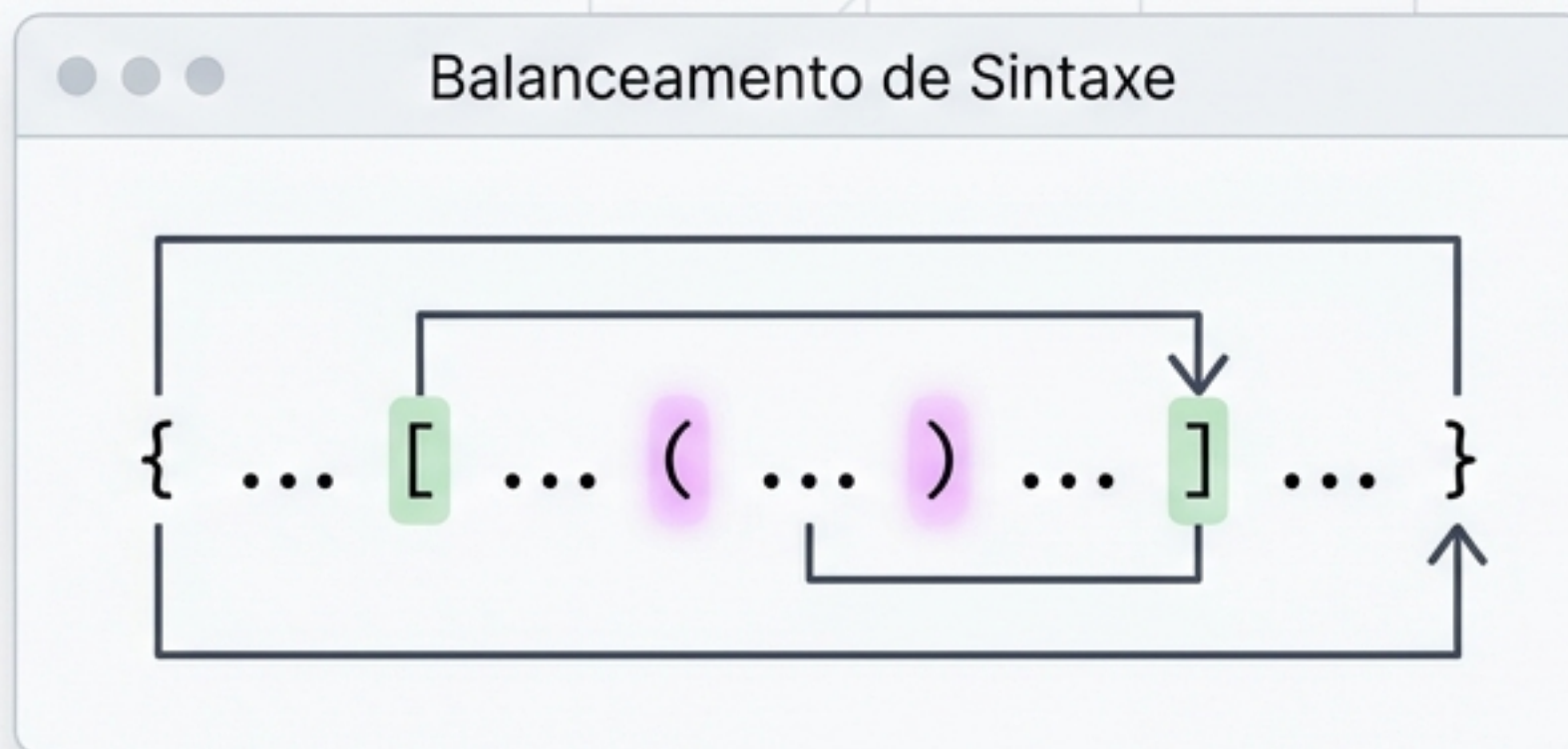
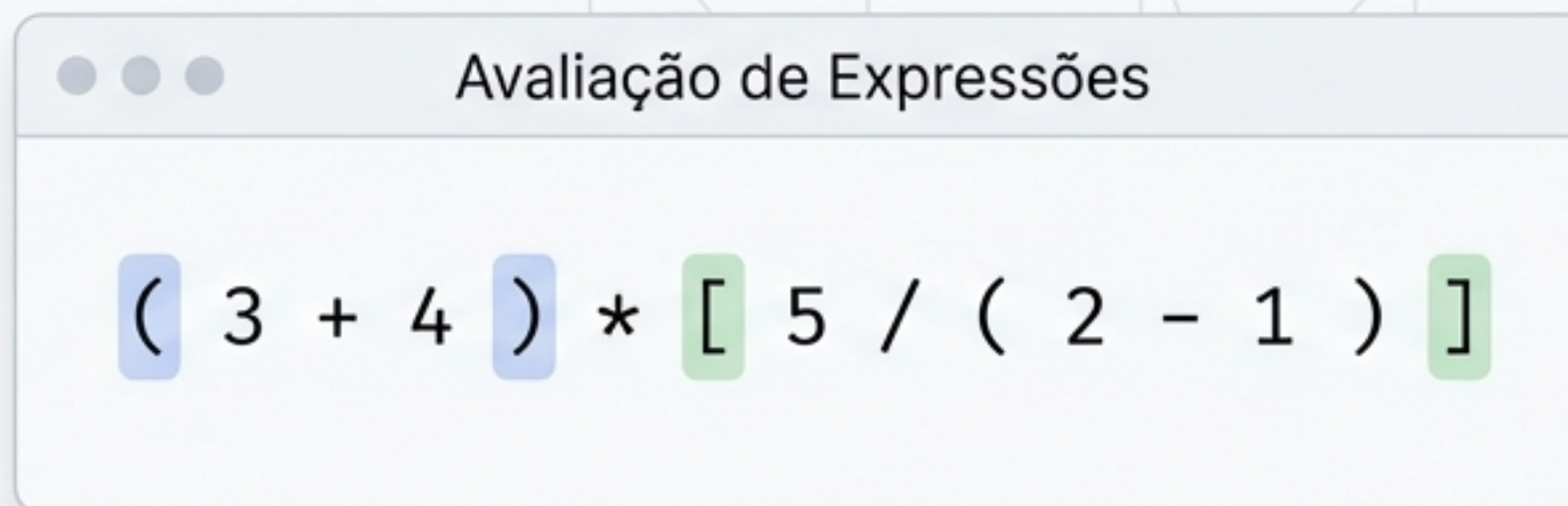
Por que Pilhas são Fundamentais?

Simplicidade & Eficiência



Redução Drástica na Complexidade de Tempo e Espaço

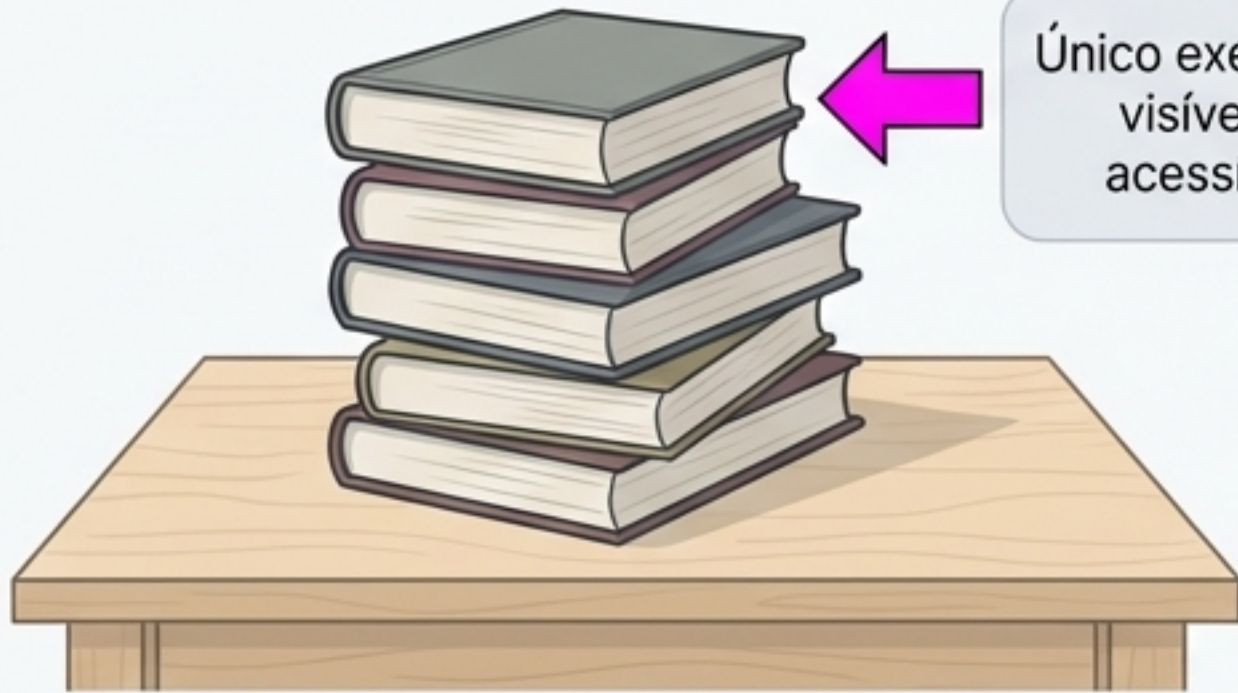
Estruturas de dados poderosas que transformam problemas algorítmicos intratáveis em resoluções lineares e elegantes.



O Princípio LIFO: Last-In, First-Out

Analogy Mapping

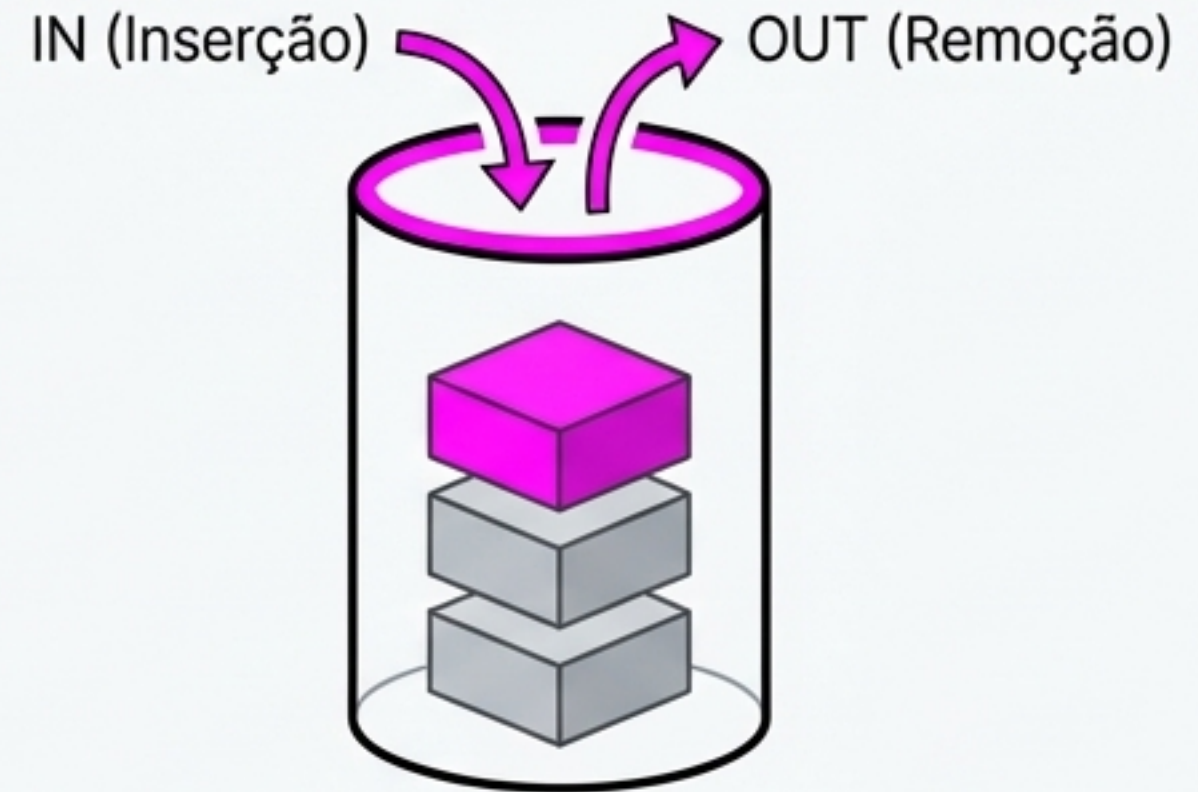
Mundo Físico



Único exemplar visível e acessível

Para alcançar o livro de baixo, todos os de cima devem ser retirados um a um.

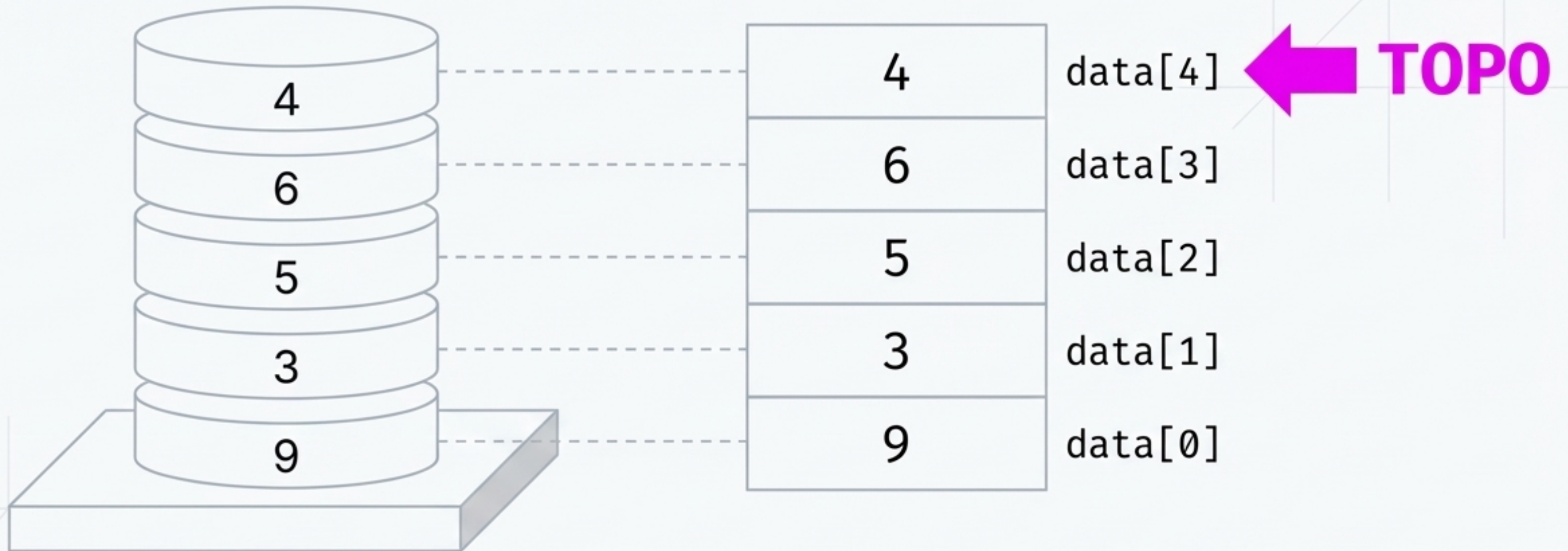
Mundo Computacional



LIFO Engine Diagram

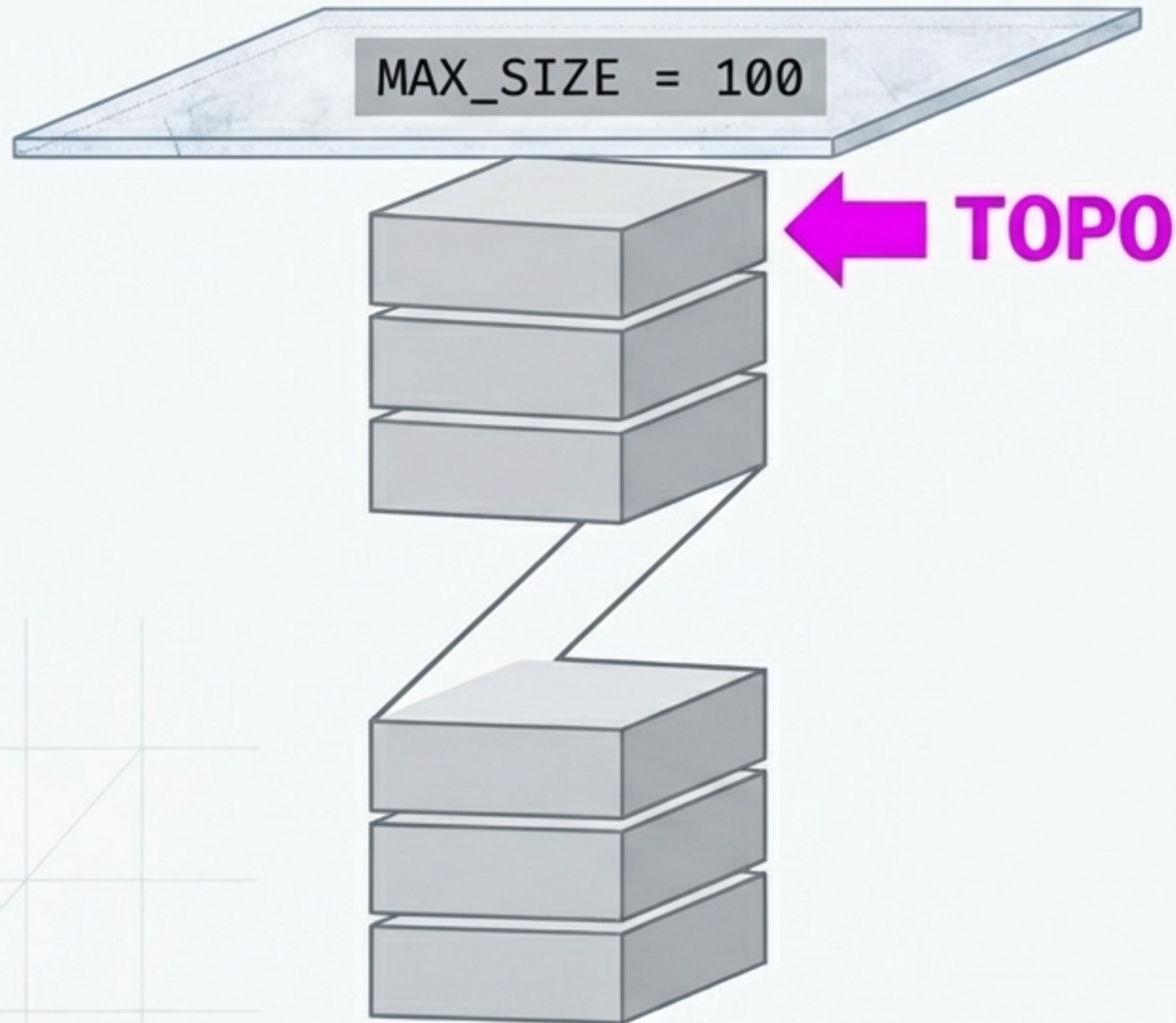
Regra Absoluta: A restrição fundamental exige que inserções e remoções sejam executadas exclusivamente em uma única posição referenciada como TOPO.

Arquitetura de Dados: O Vetor Vertical



Em C, a pilha física é planejada em um vetor contínuo de memória, controlado por um único índice numérico inteiro.

O Limite de Capacidade



A Restrição Física

Vetores em C possuem tamanho fixo alocado na memória.

A Condição de Erro

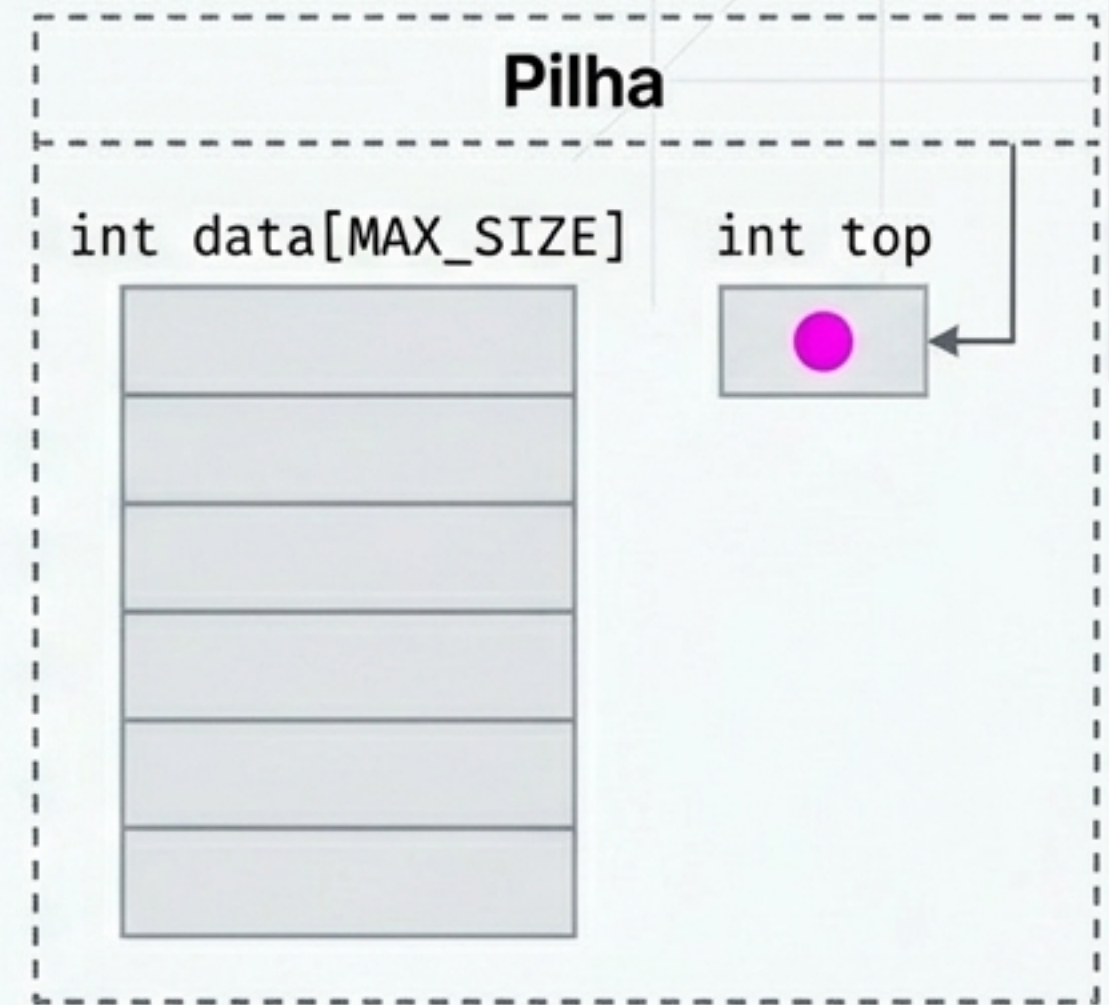
Antes de qualquer operação de inserção, o sistema deve verificar rigorosamente se o TOPO colidiu com o limite da capacidade, prevenindo um erro fatal de Stack Overflow.

Preparando a Estrutura em C

```
#define MAX_SIZE 100

typedef struct {
    int data[MAX_SIZE];
    int top;
} Pilha;
```

Anatomia da Struct



A pilha é um Tipo Abstrato de Dados (TAD) encapsulado. Um ponteiro para esta estrutura será passado por valor para todas as funções de manipulação.

Geometria de Ponteiros: O Estado Vazio



```
return pilha->top == -1;
```

O Rastreador
de Índice

Operador de
Comparação
Lógica

Posição nula,
fora dos limites
do vetor

Retorna 1 (Verdadeiro) se vazia, 0 (Falso)
se houver elementos armazenados.

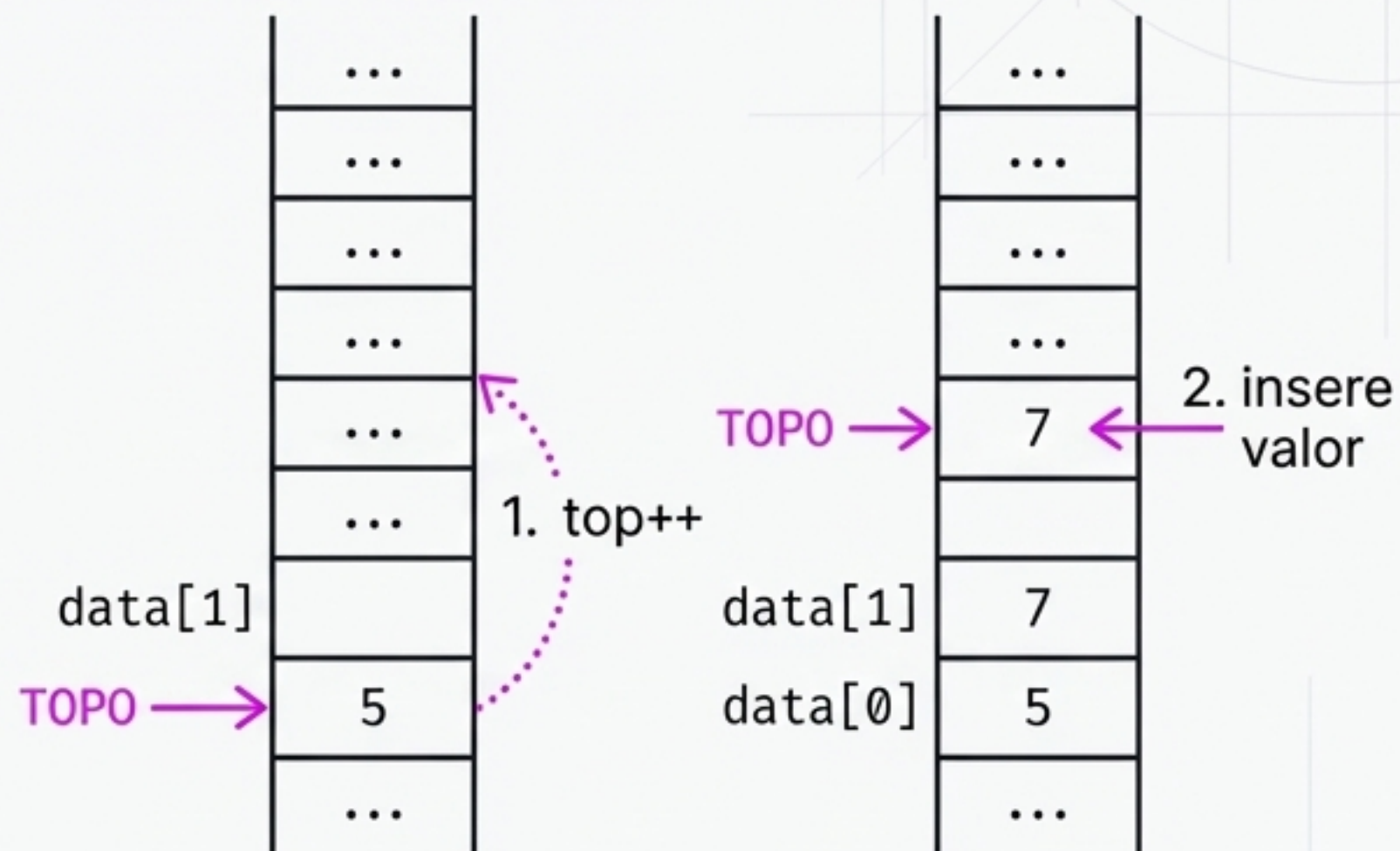
Matriz de Operações Essenciais

Operação	Ação no TOPO	Pré-condição (Erro)	Retorno
PUSH (Inserir)	+ 1 (Sobe)	Pilha não está CHEIA	void
POP (Remover)	- 1 (Desce)	Pilha não está VAZIA	void
TOP (Consultar)	0 (Imóvel)	Pilha não está VAZIA	int (0 Dado)

Nota Arquitetural: Todas as abordagens de implementação utilizadas para listas genéricas são válidas aqui, porém rigidamente restritas por estas três regras de acesso.

Operação: PUSH (Adicionar)

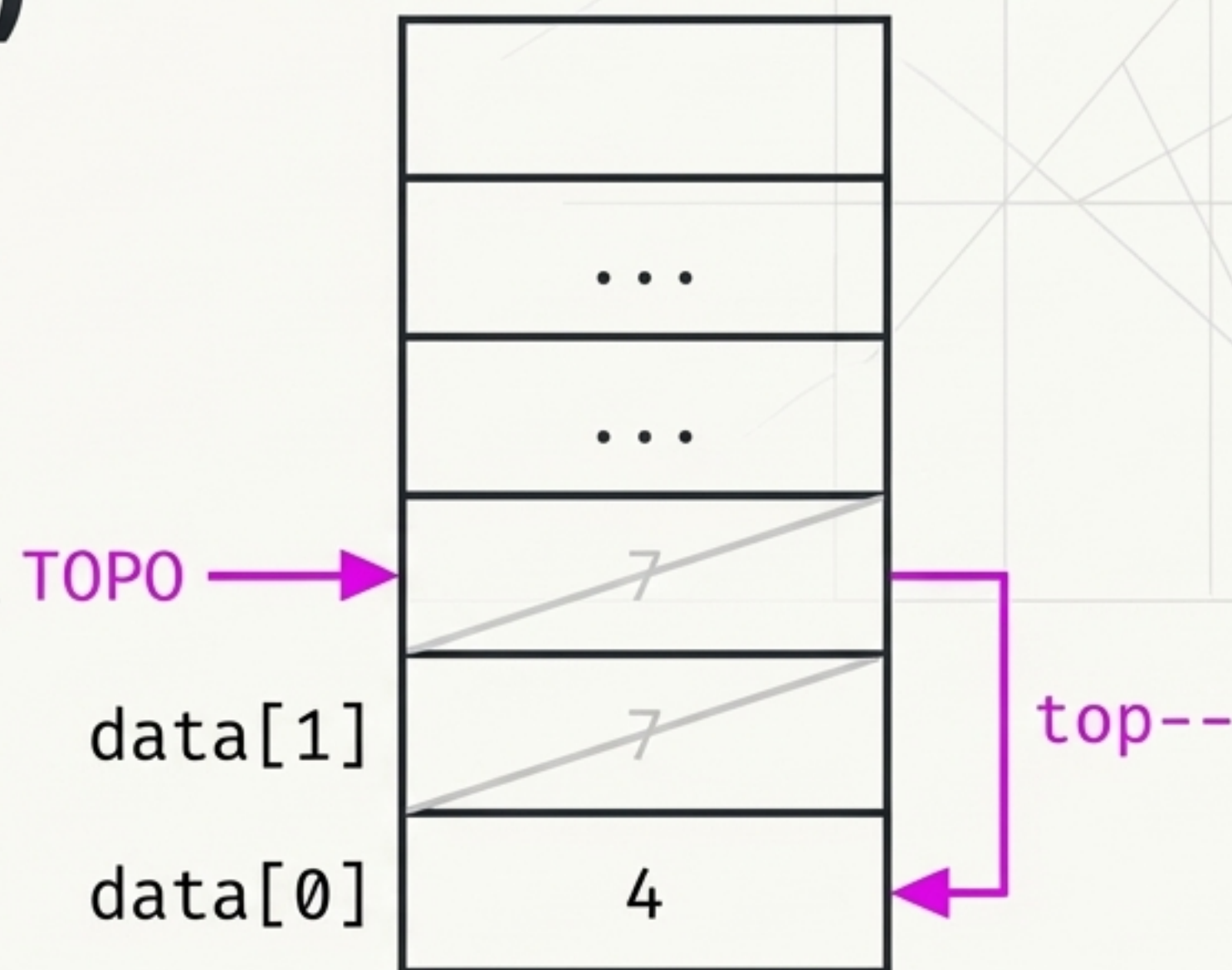
```
void push(Pilha* pilha, int valor) {  
    if (pilha_cheia(pilha)) return;  
    pilha->top++;  
    pilha->data[pilha->top] = valor;  
}
```



Regra de Ouro: O incremento do ponteiro TOPO sempre precede a inserção física do dado na memória.

Operação: POP (Remove)

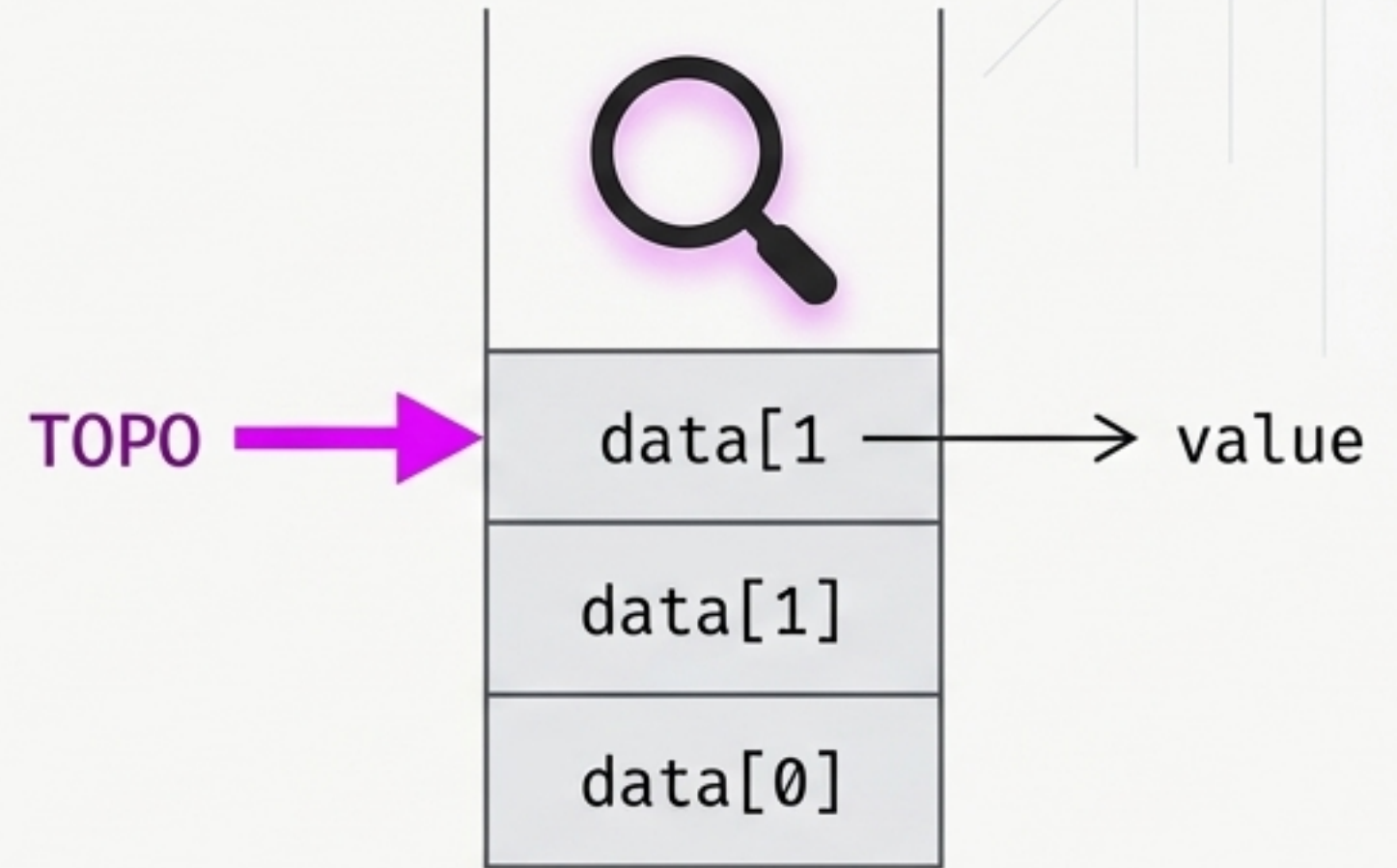
```
void pop(Pilha* pilha) {  
    if (pilha_vazia(pilha)) return;  
    pilha->top--;  
}
```



Detalhe Crítico: Em vetores, não 'apagamos' o dado da memória. Apenas rebaixamos o TOPO, invalidando o acesso ao elemento superior, que será eventualmente sobrescrito.

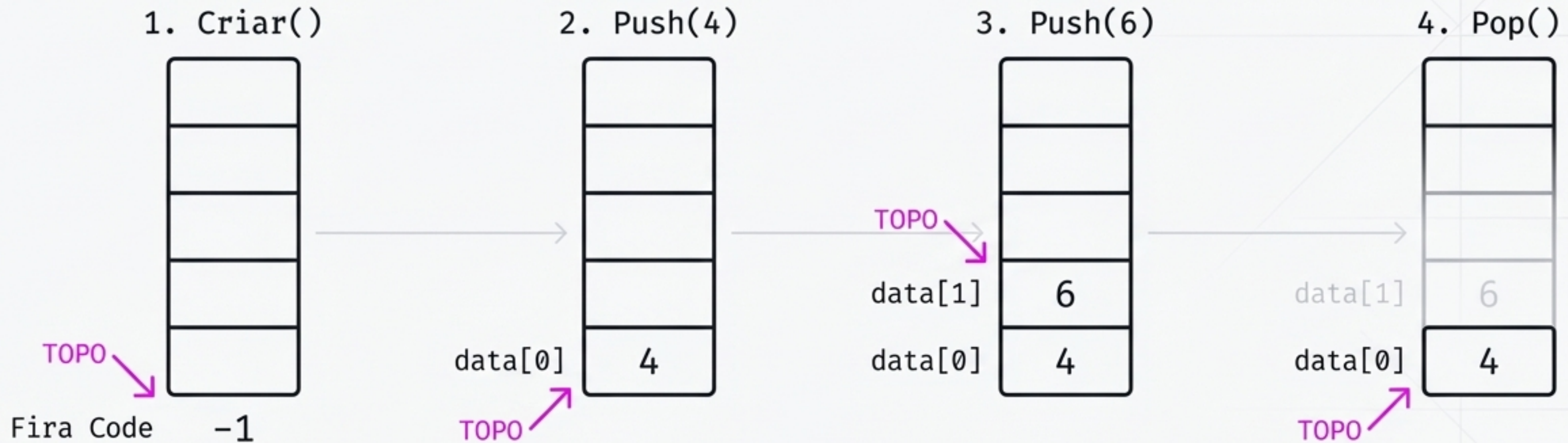
Operação: TOP (Consultar)

```
int top(Pilha* pilha) {  
    if (pilha_vazia(pilha)) return -1;  
    return pilha->data[pilha->top];  
}
```



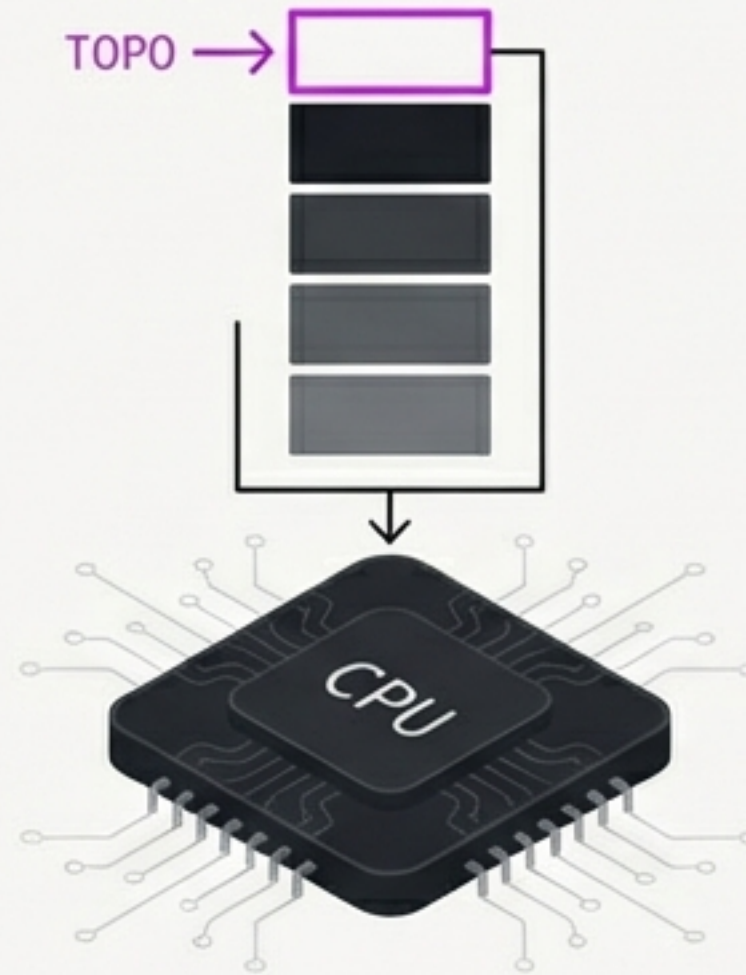
Acesso em tempo constante $O(1)$.
Retorna o valor mais recente sem alterar
a integridade estrutural da pilha.

Síntese do Sistema: Ciclo de Vida



O controle absoluto de estado reside puramente na manipulação de um único **número inteiro: o índice do TOPO.**

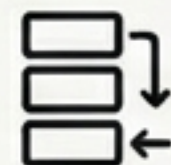
Uma Abstração Fundamental



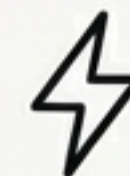
Pilhas não são apenas estruturas de código virtuais. Máquinas modernas possuem operações nativas sobre pilhas embutidas diretamente no seu conjunto de instruções de hardware (Instruction Set).



Base para Compiladores
e Parsers.



Motor do Gerenciamento
de Memória (Call Stack).



Eficiência Estrutural
Nativa $O(1)$.